

Honors Report- Spring 2026

Snigdha STP

Roll No: 2023102036

Table of Contents

1	Implementing Kalman Filtering	3
2	Velocity Estimation and Estimating Obstacles from Camera	3
3	Ensuring Stability with Lyapunov Control	4
4	Observations and Results	6
5	Fixing the Tangential Motion of the Robot	6
6	Barrier Lyapunov Based Control	7
7	Motion Scenarios and Navigation Strategy	8
7.1	Goal Seeking Motion	8
7.2	Obstacle-Aware Navigation	8
7.3	Adaptive Gain Modulation	9
7.4	Reactive Steering and Safety Control	9
7.5	Non-Holonomic Motion Enforcement	9
8	Exploring Zero-Knowledge Robot Navigation	9
8.1	State Estimation and Localization	10
8.2	Probabilistic Target Representation	10
8.3	Vision-Based Obstacle Detection	10
8.4	Lyapunov and Control Barrier Function Based Navigation	10
8.5	Operational Phases	11
9	LiDAR Based Autonomous Navigation	11
10	Implementing the Reinforcement Learning Framework	11
11	Learning-Based Navigation using Imitation Learning	12
11.1	Problem Formulation	12
11.2	Multi-Modal Input Representation	12
	Visual Input	12
	LiDAR Input	13
	Goal Representation	13
11.3	Expert Demonstration Generation	13
11.4	Dataset Construction	13
11.5	Neural Network Architecture	13
	Image Encoder	13
	LiDAR Encoder	14
	Goal Encoder	14
	Feature Fusion	14
	Actor Network	14
	Critic Network	14
11.6	Training Methodology	14
11.6.1	Policy Deployment	14

12 Performance and Observations	15
13 Links	15
13.1 Google Drive link of the ppt	15
13.2 the canva link with videos of simulations	15
13.3 Github link	15
14 References	15

1 Implementing Kalman Filtering

In autonomous robotic systems, sensor readings are often affected by noise and uncertainty. To obtain reliable localization and motion estimation, a Kalman Filter based probabilistic estimation framework was implemented.

The robot state is represented as:

$$x = [x, y, v_x, v_y]^T$$

where (x, y) denotes the robot position and (v_x, v_y) represents the velocity components.

A constant velocity motion model is used to predict the robot state:

$$x_{k+1} = Fx_k + w_k$$

where:

- F is the state transition matrix
- w_k represents process noise

The Kalman Filter continuously performs prediction and correction steps to estimate the most probable robot state from noisy sensor observations.

Two estimation configurations were implemented:

- Position-only estimation:

$$z = [x, y]$$

- Full-state estimation:

$$z = [x, y, v_x, v_y]$$

The second configuration improves responsiveness and estimation accuracy by directly incorporating velocity information.

The Kalman gain used for state correction is given by:

$$K = PH^T(HPH^T + R)^{-1}$$

which determines the contribution of prediction and measurement uncertainty during state updates.

2 Velocity Estimation and Estimating Obstacles from Camera

Velocity estimation was performed using a perception-driven approach instead of direct numerical differentiation. This better reflects real robotic systems, where direct velocity measurements are not always available.

A camera mounted on the robot captures RGB images of the environment. The captured frames are converted to grayscale and processed using the Farneback optical flow algorithm to estimate motion between consecutive frames.

The velocity components are computed from the average pixel displacement:

$$v_x = \alpha \cdot \text{mean}(\Delta x), \quad v_y = \alpha \cdot \text{mean}(\Delta y)$$

where:

- $\Delta x, \Delta y$ represent optical flow displacements
- α is a scaling factor converting image motion to real-world velocity

This enables vision-based velocity estimation similar to visual odometry techniques used in autonomous systems. The estimated velocity is fused with noisy measurements and provided as input to the Kalman Filter for improved state estimation.

Obstacle dimensions are estimated using image processing and camera geometry principles. Obstacles are first segmented using HSV color thresholding, after which contours and bounding boxes are extracted.

The real-world obstacle width is computed using the pinhole camera model:

$$\text{width}_{real} = \frac{w_{px} \cdot d}{f}$$

where:

- w_{px} is the obstacle width in pixels
- d is the estimated obstacle distance
- f is the camera focal length

The focal length is estimated from the camera field-of-view:

$$f = \frac{W/2}{\tan(FOV/2)}$$

where W denotes the image width in pixels.

The estimated obstacle dimensions and vision-based velocity measurements improve environmental awareness and enhance the robustness of the overall navigation system.

3 Ensuring Stability with Lyapunov Control

The primary navigation controller is designed using Lyapunov stability theory to guarantee stable convergence toward the target position.

An energy-based Lyapunov function is defined as:

$$V(x, v) = \frac{1}{2} \|x - x_g\|^2 + \frac{1}{2} \|v\|^2$$

where:

- x is the current robot position
- x_g is the goal position
- v is the robot velocity

The first term represents the position error relative to the target, while the second term represents the kinetic energy of the robot.

The control input is formulated as a damped proportional controller:

$$u = K_p(x_g - x) - K_d v$$

where:

- K_p controls attraction toward the goal
- K_d provides damping to suppress oscillations

The controller ensures smooth motion while reducing instability during navigation. The derivative of the Lyapunov function becomes:

$$\dot{V} = -K_d \|v\|^2 \leq 0$$

which guarantees that the system energy decreases continuously over time. Consequently:

- Velocity converges to zero
- Position converges toward the goal
- The system remains globally asymptotically stable

To perform safe navigation, obstacle avoidance forces are integrated into the controller. These include:

- Repulsive forces to move away from obstacles
- Tangential forces to move smoothly around obstacles

The repulsive force is modeled as:

$$F_{rep} \propto \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2}$$

where d is the obstacle distance and d_0 is the safety threshold.

A tangential velocity component is additionally introduced such that:

$$\nabla V \cdot v_{tan} = 0$$

This allows the robot to move along safe contours around obstacles without increasing system energy, thereby reducing the possibility of local minima trapping.

To maintain physically feasible motion, velocity saturation is applied:

$$u = \frac{u}{\|u\|} \cdot u_{max}$$

which limits velocity magnitude while preserving motion direction and maintaining stability guarantees.

4 Observations and Results

Experimental analysis demonstrates that the proposed framework provides stable and reliable autonomous navigation performance.

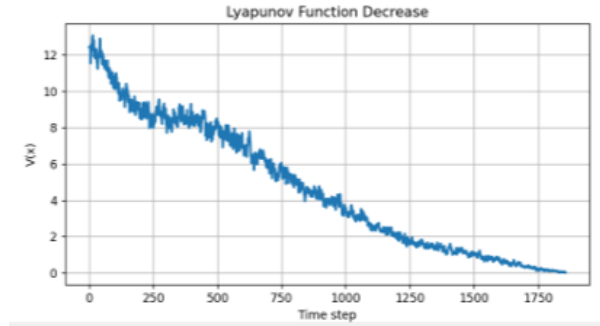
The Kalman Filter based sensor fusion framework significantly improves localization accuracy and state estimation robustness under noisy sensing conditions. Vision-based velocity estimation remains stable over time and provides realistic motion estimation without requiring dedicated velocity sensors.

The obstacle dimension estimation framework successfully approximates real-world object dimensions, improving obstacle-aware navigation and collision avoidance performance.

The Lyapunov-based controller produces:

- Smooth convergence toward the goal
- Stable motion without oscillations
- Safe obstacle avoidance behavior
- Continuous reduction in system energy

Experimental plot shows a monotonic decrease in the Lyapunov function during navigation, validating the theoretical stability guarantees of the proposed control framework.



5 Fixing the Tangential Motion of the Robot

To ensure realistic motion behavior, the tangential or sideways motion of the robot was constrained using a non-holonomic vehicle model. In practical car-like robots, lateral movement is physically infeasible since the wheels allow motion primarily in the forward direction while changing orientation through steering.

To address this, the control framework was designed such that the robot moves only using forward linear velocity and steering angular velocity. Any unwanted tangential velocity component was eliminated during control computation, ensuring that the generated trajectories satisfy non-holonomic constraints.

Additionally, the navigation system combines attractive forces toward the goal with repulsive forces away from nearby obstacles. This allows the robot to reach the desired target while safely avoiding collisions in the environment.

A Lyapunov-based control law was further implemented to guarantee stable convergence of the robot toward the goal position. The controller ensures that the tracking error decreases continuously over time, resulting in smooth and stable motion.

For robust localization and state estimation, a Kalman Filter based estimation framework was used. The filter combines sensor observations with the system dynamics to obtain accurate estimates of the robot states while reducing the effect of measurement noise. Ground truth data was used to validate the estimation performance and overall system accuracy.

6 Barrier Lyapunov Based Control

To ensure safe and stable navigation in the presence of obstacles and state constraints, a Barrier Lyapunov Function (BLF) based control approach was incorporated into the control framework. Unlike conventional Lyapunov methods that only guarantee stability, the Barrier Lyapunov approach additionally ensures that certain system states remain within predefined safe limits.

The main objective of the Barrier Lyapunov controller is to prevent the robot states from violating safety constraints while still guaranteeing convergence toward the desired goal. This is achieved by introducing a barrier function into the Lyapunov formulation. As the system state approaches a constraint boundary, the barrier function grows rapidly, thereby generating strong corrective control actions that prevent the state from crossing unsafe regions.

For the mobile robot navigation problem, the Barrier Lyapunov controller was used to:

- Maintain safe distance from obstacles
- Restrict excessive orientation and tracking errors
- Ensure stable convergence toward the target position
- Prevent violation of non-holonomic motion constraints

The controller combines attractive forces toward the goal with repulsive effects generated near obstacle boundaries. As the robot approaches an obstacle, the barrier component increases significantly, forcing the robot trajectory away from unsafe regions while preserving system stability.

Mathematically, the Barrier Lyapunov Function can be represented as:

$$V(e) = \frac{1}{2} \log \left(\frac{1}{1 - \left(\frac{e}{e_{\max}} \right)^2} \right)$$

where:

- e represents the tracking or state error
- $e_{\max}$ represents the allowable bound on the error

As the error approaches the boundary value $e_{\max}$, the Lyapunov function tends toward infinity, thereby preventing the system from violating the prescribed constraint.

The overall control strategy ensures that the robot follows physically feasible trajectories using only forward velocity and steering motion, while simultaneously achieving obstacle avoidance, robust stability, and safe navigation.

7 Motion Scenarios and Navigation Strategy

The proposed navigation framework operates under multiple motion scenarios depending on the proximity and orientation of nearby obstacles. The controller combines Lyapunov-based goal attraction, barrier-based obstacle avoidance, and adaptive gain modulation to ensure smooth, stable, and collision-free motion.

7.1 Goal Seeking Motion

In the absence of nearby obstacles, the robot follows a Lyapunov-based attractive control law to move toward the target position. The controller computes the position tracking error as:

$$e_x = x_{goal} - x, \quad e_y = y_{goal} - y$$

where (x, y) represents the current robot position and (x_{goal}, y_{goal}) denotes the target location.

The control gains are dynamically adjusted using the neural network based perception module. Visual and sensor features are processed to generate adaptive controller parameters, resulting in smoother motion and improved convergence performance.

The overall control pipeline is:

Sensor Input $\rightarrow$ Feature Extraction $\rightarrow$ Neural Network $\rightarrow$ Adaptive Gains $\rightarrow$ Lyapunov Controller

This enables stable goal-directed navigation while minimizing oscillations and tracking error.

7.2 Obstacle-Aware Navigation

When obstacles are detected near the robot, the controller augments the attractive control action with barrier-based repulsive and tangential components.

A safety function is defined as:

$$h(x) = d - R$$

where:

- d is the distance between the robot and the obstacle
- R is the safety radius around the robot

As the robot approaches an obstacle, the barrier function generates a repulsive effect that pushes the robot away from unsafe regions. Simultaneously, a tangential motion component is introduced to guide the robot smoothly around the obstacle instead of stopping directly in front of it.

The navigation flow becomes:

Obstacle Detection $\rightarrow$ Barrier Force Computation $\rightarrow$ Tangential Steering $\rightarrow$ Combined Motion Control

This results in smooth obstacle avoidance while maintaining progress toward the goal.

7.3 Adaptive Gain Modulation

To improve stability near obstacles, the controller adaptively modulates the control gains based on obstacle proximity. As the robot enters a caution zone, the controller gradually reduces forward aggressiveness and increases damping.

The adaptive process consists of:

Distance Measurement $\rightarrow$ Proximity Evaluation $\rightarrow$ Gain Adjustment $\rightarrow$ Control Update

This adaptive behavior reduces oscillations, improves stability, and ensures safer navigation in cluttered environments.

7.4 Reactive Steering and Safety Control

When an obstacle is detected directly ahead of the robot, the controller reduces the forward attractive force and increases steering influence to search for a collision-free direction.

If the obstacle becomes critically close, the controller activates an emergency safety mechanism that overrides all control actions and immediately stops the robot:

$$v = 0, \quad \omega = 0$$

where:

- v is the linear velocity
- ω is the angular velocity

This emergency stopping condition acts as the final safety layer and prevents collisions when all other avoidance strategies become insufficient.

7.5 Non-Holonomic Motion Enforcement

The robot motion model follows non-holonomic constraints corresponding to realistic car-like motion. The robot is restricted to movement through forward linear velocity and steering angular velocity only.

Sideways or tangential sliding motion is eliminated during control computation, ensuring that all generated trajectories remain physically feasible and dynamically consistent.

8 Exploring Zero-Knowledge Robot Navigation

The proposed framework performs autonomous navigation in a zero-knowledge environment, where the robot has no prior map information and does not rely on predefined environmental knowledge. The robot estimates all required states online using onboard sensing and probabilistic estimation techniques.

Unlike conventional navigation systems that depend on ground truth localization or pre-mapped environments, the robot continuously estimates its own position, velocity, and orientation directly from sensor observations.

8.1 State Estimation and Localization

Robot localization is achieved using a combination of:

- Gyroscope (IMU) measurements for orientation estimation
- Visual Odometry for motion and displacement estimation
- Kalman Filter based sensor fusion for robust state estimation

The Kalman Filter combines prediction and measurement updates to maintain uncertainty-aware estimates of the robot state. Since no ground truth information is available, estimation uncertainty naturally accumulates over time, especially during long navigation sequences.

8.2 Probabilistic Target Representation

The navigation framework maintains a probabilistic belief of the target location. As the robot gathers new observations, the target belief is continuously updated and refined.

This allows the robot to operate effectively even when the target is partially observable or temporarily occluded.

8.3 Vision-Based Obstacle Detection

Obstacle detection and tracking are performed entirely using vision-based perception without relying on a pre-built map.

The system performs:

- Obstacle detection from visual input
- Continuous obstacle tracking
- Dynamic obstacle-aware motion planning

This enables the robot to navigate safely in previously unseen environments.

8.4 Lyapunov and Control Barrier Function Based Navigation

For safe and stable navigation, the controller combines:

- Lyapunov-based attractive control for goal convergence
- Control Barrier Function (CBF) based obstacle avoidance

The Lyapunov controller ensures stable motion toward the target, while the barrier function prevents the robot from entering unsafe regions around obstacles.

8.5 Operational Phases

The navigation system operates in two major phases:

1. **Search Phase:** The robot explores the environment while attempting to detect the target using onboard perception.
2. **Navigation Phase:** Once the target is detected, the robot transitions into goal-directed navigation while performing continuous obstacle avoidance and safe motion control.

9 LiDAR Based Autonomous Navigation

A fully perception-based autonomous navigation framework was implemented without using any ground truth information in the control loop. The robot estimates its state using sensor fusion from the IMU (gyroscope) and wheel encoder measurements through an Extended Kalman Filter (EKF).

For environment perception, the system performs real-time LiDAR scanning and obstacle detection using raycasting techniques. The detected obstacle information is integrated with the Lyapunov-based navigation controller to enable safe and stable motion in unknown environments while avoiding collisions.

10 Implementing the Reinforcement Learning Framework

The reinforcement learning based navigation framework begins by capturing an RGBA image from the robot's front-facing camera. The captured image is passed through a feature extraction pipeline to obtain a compact representation of the environment.

The feature extractor converts the input image into grayscale and RGB representations and computes a 16-dimensional state vector containing:

- Goal-related features such as:
 - Distance to the target
 - Relative goal angle
 - Visibility of the target
- Obstacle-related features including:
 - Obstacle proximity
 - Angular distribution across image regions
 - Density of occupied or dark regions
- Motion-related features using optical flow estimation, representing the average pixel displacement between consecutive image frames

The extracted feature vector represents the robot’s perception of its surroundings and is provided as input to a neural network module. The network performs a forward pass to generate adaptive controller gains, namely proportional gain (K_p) and derivative gain (K_d).

During training, the neural network parameters are optimized using a policy-gradient based reinforcement learning approach. The reward function is designed to encourage:

- Efficient goal reaching
- Collision avoidance
- Smooth and stable motion

In the online control loop, the extracted features are continuously passed through the trained model to obtain updated controller gains. These gains are then used within a PD control framework to compute the robot velocity commands, enabling adaptive and environment-aware navigation.

11 Learning-Based Navigation using Imitation Learning

To improve the adaptability and generalization capability of the navigation system, an imitation learning framework was developed. The objective of this approach is to transfer the behavior of an expert-designed controller into a neural network policy capable of performing autonomous navigation. This combines the stability and safety properties of classical Lyapunov-based control with the flexibility of deep learning methods.

11.1 Problem Formulation

The navigation problem is formulated as learning a policy that maps observations to control actions:

$$\pi(o_t) \rightarrow a_t$$

where:

- o_t represents the sensory observations at time step t
- $a_t = [v, \omega]$ represents the linear and angular velocity commands

The goal is to learn a policy that can generate safe and goal-oriented navigation commands directly from sensory inputs.

11.2 Multi-Modal Input Representation

The proposed framework uses multiple sensor modalities to provide comprehensive environmental understanding.

Visual Input Front-facing camera images are processed as stacked grayscale frames. These images provide information about scene structure, free space, and obstacle layout.

LiDAR Input LiDAR scans provide distance measurements in multiple directions, allowing accurate estimation of obstacle proximity and environmental geometry.

Goal Representation A goal vector representing the relative direction and distance to the target is included as an additional navigation input.

Together, these modalities provide complementary information:

- Vision for semantic scene understanding
- LiDAR for geometric obstacle perception
- Goal vector for navigation guidance

11.3 Expert Demonstration Generation

Training data for imitation learning is generated using the Lyapunov-based expert controller. The expert control action is composed of three components:

$$u = u_{goal} + u_{avoid} + u_{tangent}$$

where:

- u_{goal} drives the robot toward the target
- u_{avoid} produces obstacle repulsion
- $u_{tangent}$ enables smooth motion around obstacles

The robot is simulated in multiple environments containing static and dynamic obstacles, and the generated state-action pairs are recorded as expert demonstrations.

11.4 Dataset Construction

The collected dataset contains:

- Camera image frames
- LiDAR scan measurements
- Goal vectors
- Expert control actions

These samples capture a wide range of navigation situations and are used to train the imitation learning model.

11.5 Neural Network Architecture

The learning framework follows a multi-modal Actor-Critic architecture.

Image Encoder A Convolutional Neural Network (CNN) extracts spatial and structural information from image inputs.

LiDAR Encoder Fully connected layers process LiDAR distance measurements and obstacle-related features.

Goal Encoder A separate fully connected network encodes the relative goal position and direction.

Feature Fusion The encoded representations are combined into a shared latent feature space:

$$z = f_{fusion}(z_{img}, z_{lidar}, z_{goal})$$

Actor Network The actor network predicts continuous control commands corresponding to linear and angular velocities.

Critic Network The critic network estimates the value function $V(s)$ to improve training stability and policy learning.

11.6 Training Methodology

The policy is trained using behavioural cloning, where the objective is to imitate the expert controller actions. The training loss is defined as:

$$L = \|a_t^{pred} - a_t^{expert}\|^2$$

which minimizes the difference between predicted actions and expert demonstrations. Additional training techniques include:

- Input normalization
- Mini-batch optimization
- Action bounding using $\tanh$ activation

11.6.1 Policy Deployment

After training, the neural network replaces the expert controller during execution. At each timestep:

- Sensor observations are collected
- The trained network predicts control actions
- The predicted actions are applied directly to the robot

The robot therefore performs fully autonomous navigation without requiring expert intervention.

12 Performance and Observations

The imitation learning framework successfully reproduces expert navigation behavior and demonstrates:

- Stable and smooth navigation
- Effective obstacle avoidance
- Generalization to unseen environments
- Lower computational complexity during inference

However, the overall performance remains dependent on the quality and diversity of the expert demonstrations. In highly complex scenarios, small prediction errors may accumulate over time in the absence of corrective feedback mechanisms.

13 Links

13.1 Google Drive link of the ppt

<https://drive.google.com/file/d/18fdLBaY4fP5WHf67IgCTQTMc9p0eWnsM/view?usp=sharing>

13.2 the canva link with videos of simulations

<https://canva.link/xnj0na7x0gxroy9>

13.3 Github link

https://github.com/S1403-6/RL_project

14 References

References

- [1] P. Fiorini and Z. Shiller, *Motion Planning in Dynamic Environments Using Velocity Obstacles*, International Journal of Robotics Research, vol. 17, no. 7, pp. 760–772, 1998.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.
- [3] T. P. Lillicrap *et al.*, *Continuous Control with Deep Reinforcement Learning*, arXiv:1509.02971, 2015.